

Estrutura de Dados 2

Prof. Igor Calebe Zadi
igor.zadi@ifsp.edu.br



INSTITUTO FEDERAL
São Paulo
Campus Catanduva



II. Busca e Ordenação



II. Busca e Ordenação

1. Métodos de busca em vetores
2. Introdução à ordenação
3. Algoritmos de ordenação por seleção, inserção e troca (bolha)
4. Outros métodos de ordenação
5. Implementação e análise de complexidade

1. Busca em vetores



1. Busca em vetores

- Busca
 - Quando se quer saber se o vetor contém ou não um **elemento-chave**
 - **Três tipos de busca são possíveis**
 - **Linear**: força bruta
 - **Binária**: subdivisão recursiva do espaço de busca
 - **Proporcional**: estimativa de posição

1. Busca em vetores

- Busca linear (*linear search*)
 - vetor é suposto não-ordenado
 - neste caso, é preciso percorrer o vetor desde o seu início, até
 - encontrar o elemento-chave
 - atingir o final do vetor sem encontrar o elemento-chave

índice	→	0	1	2	3	4
conteúdo	→	15	-3	44	-10	59
chave: -3	→	? não !	? Ok !			
chave: 38	→	? não !	? não !	? não !	? não !	? não !

não Ok !

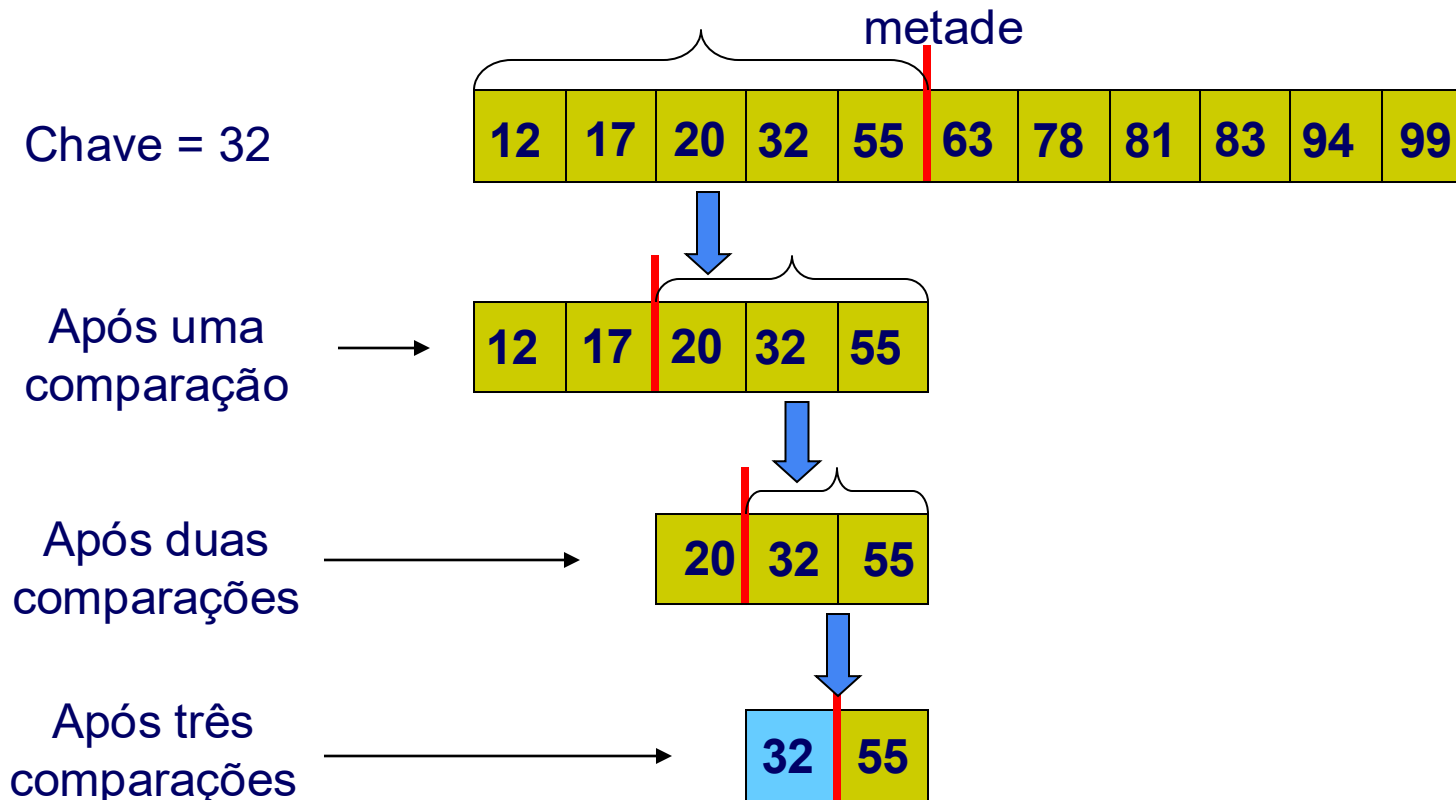


1. Busca em vetores

- Busca binária (*binary search*)
 - vetor é suposto ordenado
 - **ideia**: dividir recursivamente o espaço de busca em **duas partes**
 - comparar o valor da chave com o valor do elemento situado no meio do vetor
 - ou a chave é encontrada nessa posição
 - ou se sabe em qual das metades do espaço de busca ela pode estar (pois o vetor está ordenado)

1. Busca em vetores

- Busca binária (*binary search*) - exemplo





1. Busca em vetores

- Busca linear x busca binária
 - **linear**: exige, em média, percorrer metade do vetor até encontrar o elemento-chave
 - eficiente apenas para pequenos vetores!
 - **binária**: sendo N o tamanho do vetor, são necessárias no máximo $\log_2 N + 1$ comparações
 - praticamente **independe** do tamanho do vetor
 - porém, o vetor deve estar **ordenado** ...

1. Busca em vetores

- Busca proporcional (*proportional search*) - exemplo

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
12	40	77	101	128	163	187	212	250	284	305	333	377	401	440

$$\text{Taxa} = \frac{440 - 12}{14} = 30,6$$

$$\text{Chave} = 212 \Rightarrow \text{posição estimada} = \left\lfloor \frac{212 - 12}{30,6} \right\rfloor = \lfloor 6,54 \rfloor = 6$$

2. Ordenação de vetores



2. Ordenação de Vetores

- Torna a operação de busca mais eficiente
- Pode ser:
 - **interna**: realizada totalmente na memória principal
 - **externa** : quando utiliza também um dispositivo auxiliar (disco) para comportar os dados
 - a performance é limitada pela velocidade de acesso
 - os dados são geralmente repartidos; cada pedaço é ordenado internamente e o resultado é mesclado (intercalado)



2. Ordenação de Vetores

- Um método de ordenação é dito:
 - **estável**: se a ordem original dos elementos é preservada em caso de igualdade (“empate”) no critério de classificação
 - interessante quando se quer manter a ordem cronológica, por exemplo
 - **instável**: se não há preocupação em manter a ordem original dos elementos em caso de “empate”



2. Ordenação de Vetores

- Existem diversos métodos de ordenação interna:
 - por seleção
 - por inserção
 - por troca
 - por particionamento
 - etc ...



3. Algoritmos de ordenação

- Ordenação por seleção
 - o vetor é dividido em **dois segmentos**:
 - **1**: ordenado, está inicialmente vazio
 - **2**: **não ordenado**, contém inicialmente todos os elementos
 - **repetidamente**:
 - o elemento de menor(*) valor do **segmento 2** é selecionado e colocado **ao final do segmento 1**
 - o segmento 1 possuirá mais um elemento
 - o segmento 2 possuirá menos um elemento
- (*) ordem decrescente: inverter comparação

3. Algoritmos de ordenação

- Ordenação por seleção ([outro exemplo](#))

	ord.	não ord.									
inicialmente		14	3	22	9	10	14	2	7	25	6
após o 1º. passe		2	3	22	9	10	14	14	7	25	6
após o 2º. passe		2	3	22	9	10	14	14	7	25	6
após o 3º. passe		2	3	6	9	10	14	14	7	25	22
...											
após o 10º. passe		2	3	6	7	9	10	14	14	22	25

3. Algoritmos de ordenação

- Ordenação por seleção - observações
 - o exemplo mostra que o método é **instável**
 - para um vetor com n elementos
 - o método requer n “passes” sobre o segmento 2
 - no i -ésimo passe, fazem-se $(n - i)$ comparações
 - o total de comparações é: $\sum_{i=1}^n (n - i) = \frac{n(n-1)}{2}$
 - se o tamanho do vetor é **duplicado**, a quantidade de comparações será **quaduplicada**!
 - método tem complexidade da ordem de $O(n^2)$

3. Algoritmos de ordenação

- Ordenação por seleção - códigos

Algoritmo: ordenação por seleção recursivo
Ordena o vetor in loco

```
seleção(A, i, tam)
inicio
  se(i < tam) então
    im <- menor(A, tam, i)
```

```
    troca <- A[im]
    A[im] <- A[i]
    A[i] <- troca
```

```
    seleção(A, i+1, tam)
  fimse
```

```
fimfunção
```

```
1 Algoritmo "Selection sort"
2 //ordenar de forma crescente
3
4 Var
5 x: vetor [0..4] de inteiro
6 i, j, eleito, menor, p: inteiro
7
8 Inicio
9
10 para i <- 0 até 4 faça
11     escreval("Digite o número para: ",i)
12     leia (x[i])
13 fimpara
14
15 //percorre o vetor até a penúltima posição
16 para i <- 0 até 3 faça
17
18     eleito <- x[i]
19     //escolhe um elemento para comparar e encontrar o menor
20     menor <- x[i+1]
21     p <- i + 1
22     //percorre os demais elementos para encontrar o menor
23     para j <- i+2 até 4 faça
24         se (x[j] < menor) então
25             menor <- x[j]
26             p <- j
27         fimse
28     fimpara
29
30     //efetua a troca do eleito com o menor
31     se (menor < eleito) então
32         x[i] <- x[p]
33         x[p] <- eleito
34     fimse
35
36 fimpara
37
38 para i <- 0 até 4 faça
39     escreval ("x[",i,"] = ",x[i])
40 fimpara
41
42
43 Fimalgoritmo
```

3. Algoritmos de ordenação

- Ordenação por inserção
 - o vetor é dividido em dois segmentos:
 - **1: ordenado**, contém inicialmente apenas o primeiro elemento do vetor
 - **2: não ordenado**, contém inicialmente os demais elementos
 - **repetidamente:**
 - o primeiro elemento do segmento 2 é inserido no segmento 1 de forma ordenada
 - o segmento 1 possuirá mais um elemento
 - o segmento 2 possuirá menos um elemento

3. Algoritmos de ordenação

- Ordenação por inserção ([outro exemplo](#))

	ord.	ord.	não ord.								
inicialmente	14	3	22	9	10	14	2	7	25	6	
após o 1º. passe	3	14	22	9	10	14	2	7	25	6	
após o 2º. passe	3	14	22	9	10	14	2	7	25	6	
após o 3º. passe	3	9	14	22	10	14	2	7	25	6	
...											
após o 9º. passe	2	3	6	7	9	10	14	14	22	25	



3. Algoritmos de ordenação

- Ordenação por inserção - observações
 - dependendo de como o algoritmo (ou programa) é elaborado, o método é **estável** ou **instável**
 - o método tem complexidade da ordem de n^2 (para um vetor com n elementos)

3. Algoritmos de ordenação

- Ordenação por inserção - códigos

Algoritmo: ordenação por inserção recursivo
Ordena o vetor in loco

```
inserção(A, n){  
    se (n > 0) então  
        inserção(A, n-1)  
        i <- n  
        enquanto(i >= 1 E A[i] < A[i-1]) faça  
            troca <- A[i]  
            A[i] <- A[i-1]  
            A[i-1] <- troca  
            i <- i - 1  
        fimenquanto  
    fimse  
fimfunção
```

```
1 Algoritmo "Insertion sort"  
2  
3 Var  
4 // Seção de Declarações das variáveis  
5 x: vetor [0..4] de inteiro  
6 i, j, eleito : inteiro  
7  
8 Inicio  
9  
10 para i <- 0 até 4 faça  
11     escreval("Digite um valor")  
12     leia(x[i])  
13 fimpara  
14  
15 //percorre o vetor  
16 para i <- 1 até 4 faça  
17     eleito <- x[i]  
18     j <- i - 1  
19  
20     //percorre os elementos do segmento ordenado para encontrar  
21     //a posição do elemento eleito  
22     enquanto ((j >= 0) e (x[j] > eleito)) faça  
23         x[j+1] <- x[j]  
24         j <- j - 1  
25     fimenquanto  
26     x[j+1] <- eleito  
27 fimenquanto  
28  
29  
30 para i <- 0 até 4 faça  
31     escreval("x[",i,"] = ",x[i])  
32 fimpara  
33 |  
34 Fimalgoritmo
```



3. Algoritmos de ordenação

- Ordenação por troca
 - essencialmente, consiste em comparar os elementos do vetor, **dois a dois**
 - caso os elementos estejam fora de ordem, sua posição é trocada
 - o método é mais eficiente quando é possível identificar de maneira inteligente os elementos a serem comparados

3. Algoritmos de ordenação

- Ordenação por troca – método da bolha
 - ***bubble sort***: adota regra simples de escolha de pares de elementos para comparações e trocas
 - o vetor é dividido em dois segmentos:
 - **segmento 1**: ordenado (inicialmente vazio)
 - **segmento 2**: não ordenado (todos os elementos)
 - cada elemento do **segmento 2** é levado à sua posição adequada através de **comparações e trocas** com os elementos adjacentes (o elemento “flutua” ou “afunda”)
 - o segmento 1 possuirá mais um elemento
 - o segmento 2 possuirá menos um elemento

3. Algoritmos de ordenação

- Ordenação por troca – *bubble sort* ([outro exemplo](#))

	não ord. ...									ord.
inicialmente	14	3	22	9	10	14	2	7	25	6
após o 1º. passe	3	14	9	10	14	2	7	22	6	25
após o 2º. passe	3	9	10	14	2	7	14	6	22	25
após o 3º. passe	3	9	10	2	7	14	6	14	22	25
...	...									
após o 8º. passe	2	3	6	7	9	10	14	14	22	25

3. Algoritmos de ordenação

- Ordenação por bolha - códigos

Algoritmo: ordenação por troca recursivo
Ordena o vetor in loco

```
bolha(A, n)
inicio
    se (n > 1) então
        para i <- 0 até n-1 faça
            se (A[i] > A[i+1]) então
                troca <- A[i]
                A[i] <- A[i+1]
                A[i+1] <- troca
            fimse
        bolha(A, n-1)
    fimpara
fimse
fimfunção
```

```
1 Algoritmo "Bubble sort"
2 //ordenar de forma crescente
3 Var
4 x: vetor [0..4] de inteiro
5 i, n, aux: inteiro
6
7 Inicio
8
9 //recebe os dados para o vetor
10 para i <- 0 até 4 faça
11     escreval("Digite o número para: ",i)
12     leia (x[i])
13 fimpara
14
15 //percorrer todos elementos do vetor
16 para i <- 0 até 4 faça
17     //percorrer até a penúltima posição
18     para i <- 0 até 3 faça
19         //se for maior que o próximo efetua a troca
20         se (x[i] > x[i+1]) então
21             aux <- x[i]
22             x[i] <- x[i+1]
23             x[i + 1] <- aux
24         fimse
25     fimpara
26 fimpara
27
28 fimpara
29
30 //exibe o vetor ordenado
31 para i <- 0 até 4 faça
32     escreval ("x[",i,"] = ",x[i])
33 fimpara
34
35 Fimalgoritmo
```



3. Algoritmos de ordenação

- Ordenação por troca (*bubble sort*) - observações
 - o método é **estável**
 - o método tem complexidade da ordem de n^2
 - o pior caso ocorre quando o vetor está ordenado de **forma inversa** àquela que se deseja obter



3. Algoritmos de ordenação

- Exercícios de fixação
 - Dado o vetor:
 - $v = [5, 2, 9, 1, 5, 6]$
 - Mostre o passo a passo para ordenar o vetor utilizando os métodos de **seleção**, **inserção** e **bolha**.
 - Repita para $v2 = [29, 10, 14, 37, 13, 12, 40, 3]$



Observações sobre o material eletrônico

- O material ficará disponível na pasta compartilhada que é acessada sob convite
- O material foi elaborado a partir de diversas fontes (livros, internet, colegas, alunos etc.)
- Alguns trechos podem ter sido inteiramente transcritos a partir dessas fontes
- Outros trechos são de autoria própria
- Esta observação deve estar presente em qualquer utilização do material fora do ambiente de aulas do IFSP - Catanduva